

```
public static String nomDeCommune(Communes c, String dept, String insee){
// résultat : le nom de la commune du département dept et de numéro
//             INSEE insee (null si absente)
    Departement tabDept = c.getValue(dept);
    if(tabDept.contains(insee))
        return tabDept.getValue(insee);
    else
        return null;
}
```

```
public static void afficherGrandsDepartements(Communes c, int min){
// effet : affiche à l'ecran les numéros des départements possédant
//         plus de min communes
    Iterator<String> it = c.newIterator();
    String grandsDepts;
    while(!it.isLast()){
        if(c.getValue(it.getValue()).size() > min)
            grandsDepts += it.getValue()+" ";
        it.goForward();
    }
    System.out.println(grandsDepts);
}
```

```
public static void supprimer(Communes c, String nomCommune){
// effet : toutes les communes de nom nomCommune sont supprimées de
//         la table c
    Iterator<String> itCom = c.newIterator();
    while(!itCom.isLast()){
        Departement tabDept = c.getValue(itCom.getValue());
        Iterator<String> itDept = tabDept.newIterator();
        while(!itDept.isLast()){
            if(tabDept.getValue(itDept.getValue()) == nomCommune)
                itDept.remove();
            else
                itDept.goForward();
        }
        if(tabDept.isEmpty())
            itCom.remove();
        else
            itCom.goForward();
    }
}
```

```
public static void miseAJour(Communes c, Communes maj){
// effet : effectue la mise a jour de la table c en utilisant la
//         table maj (qui contient de nouvelles communes, ou des
//         communes qui changent de nom).
    Iterator<String> itComMaj = maj.newIterator();
    while(!itComMaj.isLast()){
        Departement tabDeptMaj = maj.getValue(itComMaj.getValue());
        if(c.contains(itComMaj.getValue())){
            Departement tabDeptC = c.getValue(itComMaj.getValue());
            Iterator<String> itTabDeptMaj = tabDeptMaj.newIterator();
            while(!itTabDeptMaj.isLast()){
                if(tabDeptC.contains(itTabDeptMaj.getValue()))
                    tabDeptC.modify(itTabDeptMaj.getValue(), tabDeptMaj.getValue(itTabDeptMaj.
                        getValue()));
            }
        }
    }
}
```

```

        else
            tabDeptC.add(itTabDeptMaj.getValue(), tabDeptMaj.getValue(itTabDeptMaj.
                getValue()))
            itTabDeptMaj.goForward();
        }
        c.modify(itComMaj.getValue(), tabDeptC);
    }else{
        c.add(itComMaj.getValue(), tabDeptMaj);
    }
    itComMaj.goForward();
}
}

```

```

public static String nomLePlusRepandu(Communes C){
// pré-requis : !C.estVide()
// résultat : le nom de la commune qui apparait le plus grand nombre
// de fois dans la table c.
    if(c.isEmpty()) return null;
    Table<String,int> nomNFois = new Table<String,int>;
    Iterator<String> itC = C.newIterator();
    Iterator<String> itD;
    Departement tabDept;
    while(!itC.isLast()){
        tabDept = c.getValue(itC.getValue());
        itD = tabDept.newIterator();
        while(!itD.isLast()){
            keyTest = tabDept.getValue(itD.getValue());
            if(nomNFois.contains(keyTest){
                int n = nomNFois.getValue(keyTest);
                nomNFois.modify(keyTest, n++);
            }else{
                nomNFois.add(keyTest, 1);
            }
            itD.goForward();
        }
        itC.goForward();
    }
    Iterator<String> itN = nomNFois.newIterator();
    String res = "";
    Int resInt = 0;
    while(!itN.isLast()){
        int valueTest = nomNFois.getValue(itN.getValue());
        if(valueTest > resInt){
            resInt = valueTest;
            res = itN.getValue();
        }
        itN.goForward();
    }
    return res;
}

```

```

public static boolean present(ArbreBinaire<Doublet> A, int deb, int fin){
// pré-requis : !A.estVide();
// resultat : vrai si l'arbre contient un doublet dont la valeur est
// comprise entre deb et fin (bornes incluses)
// faux sinon.
    if(A.isEmpty())return false;

```

```

    Iterator<Doublet> it = A.newIterator();
    int cas = 0;
    while(it.nodeType() != NodeType.FEUILLE){
        if(it.getValue().valeur > fin) cas=1;
        else if(it.getValue().valeur < deb) cas=2;
        else cas = 3;
        switch(cas){
            case 1 :
                if(it.nodeType() != NodeType.SIMPLED)
                    it.goLeft();
                else
                    return false;
                break;
            case 2 :
                if(it.nodeType() != NodeType.SIMPLEG)
                    it.goRight();
                else
                    return false;
                break;
            case 3 :
                return true;
        }
    }
    if(it.getValue().valeur > deb && it.getValue().valeur < fin)
        return true;
    return false;
}

public static boolean lineaire(ArbreBinaire<Doublet> A){
    // pré-requis : !A.estVide();
    // resultat : vrai si A ne comporte qu'une seule branche
    //              (c'est-à-dire il n'y a pas de noeud double dans A
    //              faux sinon.
    if(A.isEmpty())return false;
    Iterator<Doublet> it = A.newIterator();
    while(it.nodeType() != NodeType.FEUILLE){
        if(it.nodeType() == NodeType.SIMPLED)
            it.goRight();
        else if(it.nodeType() == NodeType.SIMPLEG)
            it.goLeft();
        else if(it.nodeType() == NodeType.DOUBLE)
            return false;
    }
    return true;
}

public static int poids(ArbreBinaire<Doublet> A){
    // pré-requis : !A.estVide()
    // resultat : le poid de l'arbre A
    if(A.isEmpty())return false;
    Iterator<Doublet> it = A.newIterator();
    return poidsAux(it);
}

public static int poidsAux(Iterator<Doublet> it){
    int poidsDroit = 0, poidsGauche = 0;
    if(it.nodeType() != NodeType.FEUILLE){

```

```

        if(it.nodeType() != NodeType.SIMPLED) {
            it.goLeft();
            poidsGauche = poidsAux(it);
            it.goUp();
        }
        if(it.nodeType() != NodeType.SIMPLEG) {
            it.goRight();
            poidsDroit = poidsAux(it);
            it.goUp();
        }
    }
    if(poidsDroit > poidsGauche)
        return poidsDroit + it.getValue().valeur;
    else
        return poidsGauche + it.getValue().valeur;
}

public static void ajouter(ArbreBinaire<Doublet> A, Doublet d){
    // pré-requis : !A.estVide()
    // effet : le doublet d est rajouté a l'arbre (sans faire appel à la
    //          fonction present()) et les champs nombreDroite sont mis à
    //          jour quand c'est nécessaire
    //          Sans effet si un doublet de valeur d.valeur déjà présent.
    if(A.isEmpty()) return false;
    Iterator<Doublet> it = A.newIterator();
    while(it.nodeType() != NodeType.BUTOIR){
        if(it.getValue().valeur > d.valeur)
            it.goLeft();
        else if(it.getValue().valeur < d.valeur)
            it.goRight();
        else if(it.getValue().valeur == d.valeur)
            return;
    }
    d.nombreDroite = 0;
    it.add(d);
    it.goRoot();
    while(it.getValue() != d.valeur){
        if(it.getValue().valeur > d.valeur)
            it.goLeft();
        else if(it.getValue().valeur < d.valeur){
            Doublet x = it.getValue();
            x.nombreDroite++;
            it.setValue(x);
            it.goRight();
        }
    }
}
}

```